

队伍编号	MC2405626
题号	(C)

基于 ARIMA 时间序列预测模型对货量预测与人员排班问题的探究 摘要

当今社会，物流需求量随着电子商务的发展而大幅增加，客户希望迅速地收到货物，分拣中心面临着更大的货量压力。本文通过建立 ARIMA 时间序列预测等模型来解决物流网络分拣中心货量预测与人员排班问题，确保分拣作业高效进行。

针对问题一：主要探究各个分拣中心未来 30 天每天每小时的货量，属于预测类问题。对未来 30 天份每天货量进行预测：首先对附件 1 与附件 2 的数据进行预处理，其次抽取 SC1 分拣中心 8-未来 30 天每天货量构造一个时间序列，通过 Python 编程将其进行可视化；然后通过 ADF 检验、相关图与自相关图确定 ARIMA (1,1,1) 时间序列预测模型，利用 SPSS 软件得到模型参数，建立模型公式之后抽取 SC12 分拣中心对其未来 30 天每天货量进行预测；最后利用相同的方法预测其余分拣中心每天货量并将结果写入结果表 1（见支撑材料）；对未来 30 天每天每小时货量进行预测：通过 Python 编程首先算出 11 月份某天某时刻的小时级别比率，然后将各小时级别比率与结果 1 中预测数据相乘得出未来 30 天每天每小时货量，最后将结果写入结果表 2（见支撑材料）。

针对问题二：主要探究在运输路线改变的条件下（依据附件 1-4 信息），对各个分拣中心未来 30 天每天每小时货量进行预测，属于预测类问题。首先将附件 3 与附件 4 的信息通过 Python 编程进行可视化，对信息匹配找出变化的路线；其次通过编程计算出附件 4 中运输路线改变后对应的每条路线的其他路线的占比情况；然后编程计算出分配后的货量并绘制出热力图直观展示货量多少，同时绘制出修正货量前后的对比图，最后计算出货量变化率；最后对之前的预测结果进行修正将结果填入结果表 3 与结果表 4（见支撑材料）。

针对问题三：主要探究在问题二预测结果的基础上求解出未来 30 天各分拣中心、各班次的出勤人数，属于优化类问题。基于问题二的预测结果建立一个纯整数规划模型，首先确定决策变量为分配的正式工与临时工的人数；再确定约束条件包括每天的货量处理完成、每个分拣中心正式工人数不超过总人数（60 人）、每人每天只能安排一个班次等；接着确定目标函数为最小化总人天数。最后通过 Python 编程确定未来 30 天每个分拣中心每个班次的出勤人数。

针对问题四：主要探究 SC60 分拣中心未来 30 天每天正式工与临时工的班次出勤计划，属于优化类问题。需要建立一个纯整数规划模型，首先从附件 1 与附件 2 中提取出 SC60 分拣中心的相关数据；然后确定决策变量为各个正式工的出勤状况（二进制变量）与雇佣的临时工人数；再确定约束条件有每名正式员工出勤率不高于 85%、每天货量处理完成、正式工连续出勤人数不超过 7 天与正式工出勤率尽量均衡等；接着确定目标函数为最小化安排的人天数。最后通过 Python 编程确定未来 30 天每名正式工及临时工的出勤计划。

关键词：ARIMA 时间序列预测模型 混合整数规划模型 线性插值 货量预测

目录

一、 问题重述	1
1.1 问题的背景	1
1.2 问题的提出	1
二、 问题分析	2
2.1 问题一的分析	2
2.2 问题二的分析	2
2.3 问题三的分析	2
2.4 问题四的分析	3
三、 模型假设	3
四、 符号说明	3
五、 模型准备	3
六、 数据预处理	5
6.1 数据清洗	5
6.2 数据聚合	5
6.3 数据提取	5
七、 模型的建立与求解	5
7.1 问题一的模型的建立与求解	5
7.1.1 基于 ARIMA 时间序列预测模型对未来 30 天每天的货量预测	5
7.1.2 对未来 30 天每天的货量预测结果求解	5
7.1.3 对未来 30 天每小时货量的预测	11
7.1.4 对未来 30 天每小时货量的预测结果求解	12
7.2 问题二的模型的建立与求解	12
7.2.1 对路线改变后各个分拣中心未来 30 天每天及每小时货量预测	12
7.2.2 路线改变后各分拣中心未来 30 天每天及每小时货量预测结果求解	12
7.3 问题三的模型的建立与求解	16
7.3.1 基于纯整数规划模型探究未来 30 天每个分拣中心每个班次出勤人数	16
7.3.2 模型的求解	19
7.4 问题四的模型的建立与求解	19
7.4.1 基于纯整数整数线性规划模型探究未来 30 天每名正式工与临时工的班次出勤计划	19
7.4.2 模型的求解	22
八、 模型的评价	22
8.1 模型的优点	22
8.2 模型的缺点	22
8.3 模型的改进	23
8.4 模型的推广	23
九、 参考文献	24

一、问题重述

1.1 问题的背景

随着电子商务的迅猛发展，物流需求大幅增加，分拣中心面临着更大的货量压力。为了准确预测货量，能合理安排人员，确保分拣作业的高效进行，减少货物积压与延误，物流网络分拣中心货量预测与人员排班至关重要。

分拣中心的货量预测是电商物流网络需要研究的重要问题，若管理者能预知各时间段需要操作的货量，可提前对资源进行安排。两个货量预测目标：①以历史货量、物流网络配置为基础，预测各个分拣中心每天的货量；②依据历史货量小时数据，预测出各个分拣中心每小时的货量。

为了在完成工作的情况下尽可能降低人员成本，依据分拣中心货量预测的人员排班是一个重要环节，分拣中心人员分为正式工（工作效率较高）与临时工（工作效率较低且成本较高），其中临时工为临时招募的成员，依据货量情况每天可任意增减。依据当前物流网络，人员安排班次与小时人效指标如下：

每个分拣中心的 6 个班次：00:00-08:00,05:00-13:00,08:00-16:00,12:00-20:00,14:00-22:00,16:00-24:00（每个成员每天只能出勤一个班次）；

小时人效指标为每人每小时完成的货量（包裹量）。正式工小时人效不超过 25 包裹/小时，临时工小时人效不超过 20 包裹/小时。

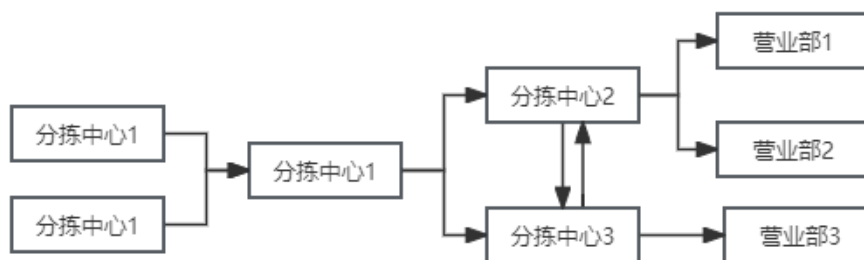


图 1：物流网络运行图

1.2 问题的提出

问题一：通过建立货量预测模型对 57 个分拣中心未来 30 天每天每小时的货量进行预测，并将预测结果写入结果表 1 与结果表 2。

问题二：假设未来 30 天分拣中心之间的运输路线改变（见附件 4），依据附件 1-4，对 57 个分拣中心未来 30 天每天和每小时的货量进行预测，并将预测结果写入表 3 与表 4。（附件三为过去 90 天各分拣中心之间的各运输线路平均货量）

问题三：假设每个分拣中心拥有 60 名正式工，人员安排时优先使用正式工，在问题二预测结果的基础上建立模型求出未来 30 天各个分拣中心、各个班次的出勤人数，并将结果写入结果表 5。在每天货量完成的基础上，要求安排的人天数（如果 25 天每天出勤 240 名员工，总人天数为 6000）尽可能少，每天实际小时人效尽量均衡。

问题四：以分拣中心 SC60 为例，假如 SC60 问题分析现有 200 名正式工，建立模型来确定将来 30 天每名正式工与临时工的班次出勤计划（给出未来 30 天每天 6 个班次中每名正式工将在哪些班次出勤，每个班次需要雇多少零时工），并将结果写入结果表 6。注意：①每名正式工的出勤率（出勤率=出勤的天数/总天数 30）应低于 85%；②连续出勤率应低于 7 天；③在每天货量处理完成的基础上，安排的人天数尽可能少、每天实际小时人效尽量均衡、正式工出勤率尽量均衡。

（附件 1：各分拣中心过去 4 个月每天的货量；附件 2：过去 30 天每小时的货量。）

二、问题分析

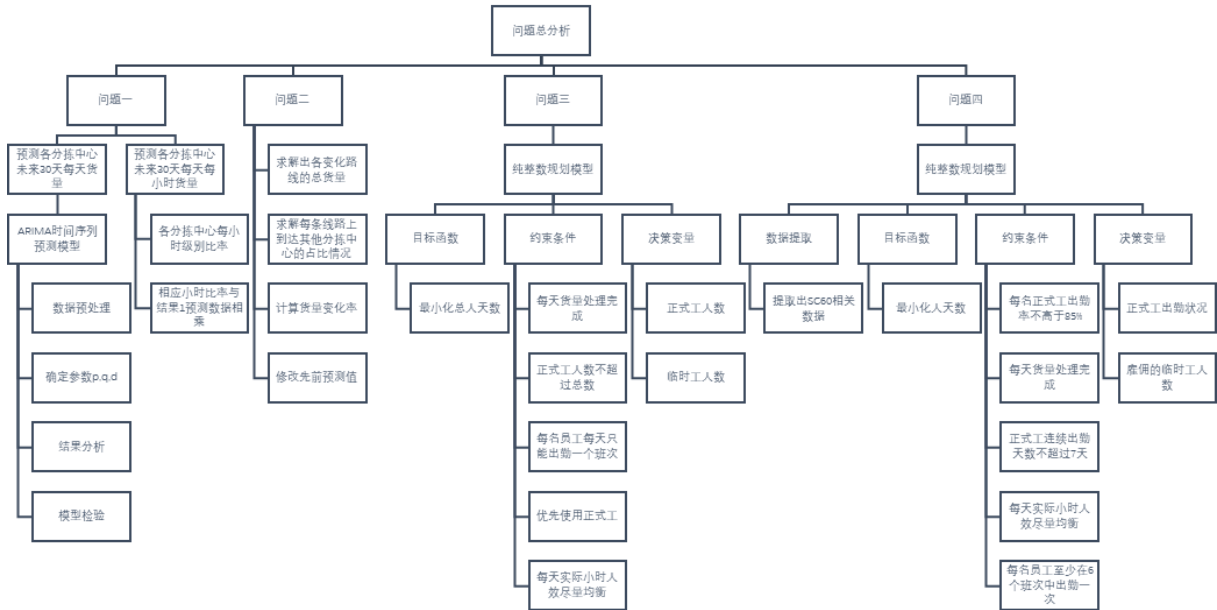


图 2：问题总分析

2.1 问题一的分析

本题需要建立货量预测模型预测出 57 个分拣中心未来 30 天每天每小时的货量。因为有时间的原因，可以考虑建立 ARIMA 时间序列预测模型来分析货量随时间变化的趋势。预测未来 30 天每天的货量：首先进行数据预处理对数据进行处理，然后构造时间序列，并判断其稳定性，若不稳定需要进行差分法将其转化为平稳性序列，再构建出 ARIMA 时间序列预测模型并对检验模型，最后预测出未来 30 天每天货量；预测未来 30 天每天每小时货量：考虑到很多传统时间序列模型无法捕捉到短频时间的相关性，可以凭借日频的预测结果推测出时频预测值。首先可以计算出每小时的级别比率，然后结合先前每天货量的预测结果，最后得出每天每小时的货量。

2.2 问题二的分析

本题需要考虑在未来 30 天分拣中心间的运输路线改变的前提下，结合附件 1-4，预测出各个分拣中心未来 30 天每天及每小时货量。可以先将附件 3 与附件 4 中的路线比较，找出变化的路线的总货量，随后按照不同比例重新将总货量分配到以该路线对应的到达分拣中心的相应的路线上，然后计算总货量的变化率，最后将问题一中对应的预测结果与之相乘，得出各个分拣中心未来 30 天每天及每小时在路线变化的前提下的预测值。

2.3 问题三的分析

本题需要以问题二预测结果为前提建立模型求出未来 30 天各个分拣中心与各班次的出勤人数。可以考虑以最小化总入天数为目标函数；以保证每天货量处理完成、确保各个分拣中心每天的工作人效尽量均衡以及各个分拣中心每天正式员工不超过总数等确定约束条件；以各分拣中心每天每班次分配的正式工人数与临时工人数为决策变量建立一个纯整数规划模型。

2.4 问题四的分析

本题需要探究当 SC60 分拣中心有 200 名正式工的前提下，建立模型确定未来 30 天每名正式工与临时工的班次出勤计划。依据问题描述的要求，依据之前的预测结果与分拣中心 SC60 的特定情况，可以考虑建立一个目标函数为最小化总人天数；约束条件为每天完成的基本货量、正式工出勤率低于 85%、正式工连续出勤天数小于 7 天等；决策变量为 0-1 变量正式工的出勤状况与雇佣的临时工人数的纯整数规划模型对正式工与临时工班次出勤作出规划。

三、模型假设

假设 1: 假设附件中的数据真实有效。

假设 2: 假设每年间快递配送量存在规律性，即可忽视年份。

假设 3: 假设线路发生变化时变化前后的平均货量不变

假设 4: 假设每年 1 月每日的货量差别不大，进行一整个月中每年的分析时大致等同于一整个月的数据进行分析。

假设 5: 假设正式工的工作模式为连续工作 5 天休息 1 天。

假设 6: 假设正式工与临时工都是健康人。

四、符号说明

符号	说明	单位
$L_{i,j}$	货量运输变化的路线	-
$Y_{i,j}$	变化的路线从分拣中心出发的平均货量	-
$Y_{n,j}$	重新分配后的总货量	-
$Y_{p,j}$	过去分拣中心的平均货量	-
$P_{n,j}$	分拣中心修正后的预测结果	-
$P_{p,j}$	分拣中心原始预测结果	-
X	正式工人数	人
Y	临时工人数	人
η_x	正式工工作效率	包裹/小时
η_y	临时工工作效率	包裹/小时
$\eta_{x,y}$	正式工与临时工实际工作效率	包裹/小时
$Y_{k,d,i}$	第 k 个分拣中心第 d 天第 i 个班次分配的临时工的人数	人
$X_{k,d,i}$	第 k 个分拣中心第 d 天第 i 个班次分配的正式工的人数	人
$L_{d,i}$	每个分拣中心第 d 天第 i 个班次的货量需求	-

五、模型准备

1. ARIMA 时间序列预测模型：

① 平稳性：要求序列的均值与方差不发生明显变化

➤ 严平稳：表示分布状况不随时间改变而改变（期望为 0，方差为 1）；

➤ 弱平稳：期望与相关系数不变（未来某时刻的取值依赖过去的信息，具有依赖性）。

② 差分法：将不平稳的时间序列转变为平稳的时间序列

时间序列在 t 时刻与 $t-1$ 时刻数值的差值（一阶差分），在一阶差分的基础上再做一次差分为二阶差分。

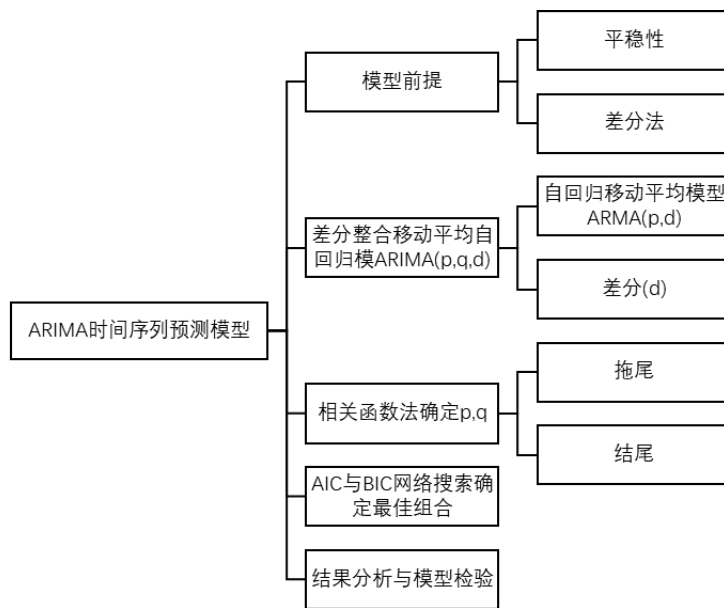


图 3：ARIMA 时间序列预测模型流程图

2. 整数规划模型^[1]:

变量全部为整数为纯整数规划；变量部分限制为整数为混合整数规划。

1) 0-1 整数规划为整数规划中的特殊情形，变量只能取 0 或 1（此时变量称为二进制变量或 0-1 变量）

2) 0-1 整数规划使用的题型有：

相互排斥的约束条件：二进制变量为 0 时采取一种方案，二进制变量为 1 时采取另一种方案；固定费用问题：有些固定费用问题不能用一般的线性规划来描述，可用混合整数规划解决；指派问题：分配固定的人做固定的工作，每人做且仅做一项工作。

3. 解决方法：

解决 0-1 整数规划问题的算法有很多，其中最常见有：①蒙特卡洛法（随机取样法），也称计算机随机模拟方法，基于大量事件的统计结果实现一些确定性问题的计算，使用蒙特卡洛法时必须使用计算机生成相关分布的随机数；②分支定界法中的分支：通常把全部可行解空间反复分割为越来越小的子集；定界：对每个子集内的解集计算一个目标下界（对于最小值问题），这称为定界。在每次分枝后，凡是界限超出已知可行解集目标值的那些子集不再进一步分枝，这样，许多子集可不予考虑，这称剪枝；③割平面法：先不考虑整数性约束，求解相应的线性规划问题。若线性规划问题的最优解恰好是整数解，则此解即为整数规划问题的最优解。否则，就增加一个新的约束条件，称为割平面。

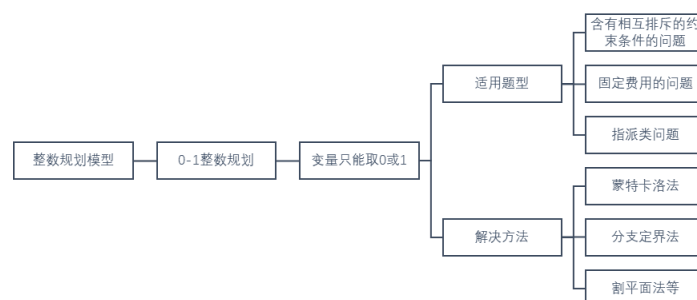


图 4：0-1 整数规划

六、数据预处理

6.1 数据清洗

1.异常值：通过 EXCEL 的查找与筛选功能发现在 11 月 2 号、11 月 11 号附近货量急剧上升，考虑到可能是由于“双十一”活动货量需求量增加的缘故，不将其视为异常值，但是考虑到可能会影响后续的预测结果，因此，本文将这些数据进行线性插值后再进行预测，提高预测进度。

2.缺失值：通过分析题目发现附件 2 中数据应为 41040 条数据，而表中数据只有 33281 条，说明某个分拣中心在某天某时刻数据缺失，例如在 SC14 分拣中心 11 月 2 日凌晨 1 点到 5 点、中午 12 点的数据缺失。本文将这些地方填补为 0。

6.2 数据聚合

1.解决问题一时需要将附件 2 中提供的过去 30 天每天 24 小时的的货量相加，与附件 1 中每天的总货量聚合分析它们之间的货量差异。

2.路线发生改变后，需要将附件 3 与附件 4 分拣中心之间路线信息聚合分析路线发生的变化。

6.3 数据提取

问题四以 SC60 分拣中心确定出勤计划，需要从附件 1 与附件 2 中提取出对应 SC60 分拣中心的数据对问题进行分析。

七、模型的建立与求解

7.1 问题一的模型的建立与求解

7.1.1 基于 ARIMA 时间序列预测模型对未来 30 天每天的货量预测

step1: 通过 Python 编程对附件 1 每个分拣中心 122 天的工作进行可视化观察工作天数是否相等（即是否有缺失值）。

step2: 通过 Python 编程对各个分拣中心的总货量与不同日期的总货量进行可视化。对附件 1 与附件 2 的数据进行对比。

step3: 通过 Python 编程对附件 2 各个分拣中心时间与记录数之间的关系进行可视化观察记录数是否相等（即是否有缺失值）。

step4: 若数据存在缺失值需要进行插值处理，并将插值后的数据相比较。

step5: 对不同日期每天货量进行可视化观察数据特征。

step6: 随机抽取几个分拣中心对他们的总货量进行可视化并绘制出相应的箱线图，依据图形进行分析。

step7: 任意选取一个分拣中心对其 4 个月每天货量进行可视化，观察图像趋势是否平稳，若不平稳需要构造平稳序列。

step8: 构建 ARIMA 时间序列预测模型并进行模型检验。

step9: 绘制出该分拣中心的时间序列预测图得出该分拣中心未来 30 天每天货量的预测值并使用该方法将剩余 56 个分拣中心未来 30 天每天的货量预测出来。

7.1.2 对未来 30 天每天的货量预测结果求解

step1: 附件 1 数据可视化：

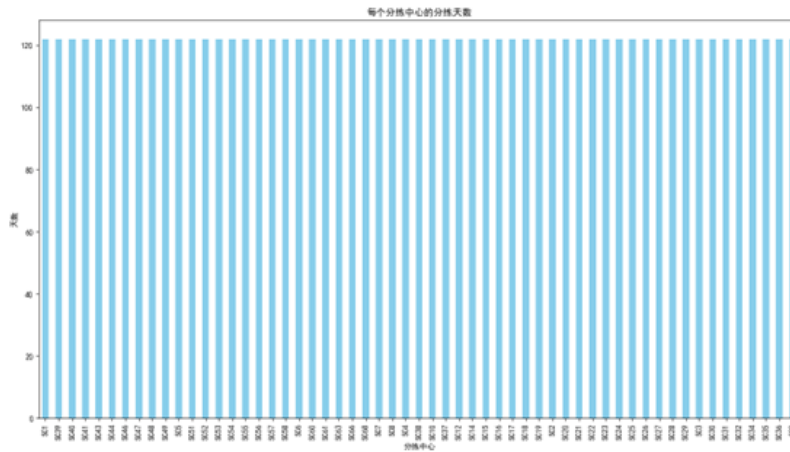


图 5：附件 1 数据可视化

由图 5 可知各个分拣中心工作天数处于同一时间段且不存在缺失值。

step2: 1.将附件 1 与附件 2 分拣中心的货量进行可视化：

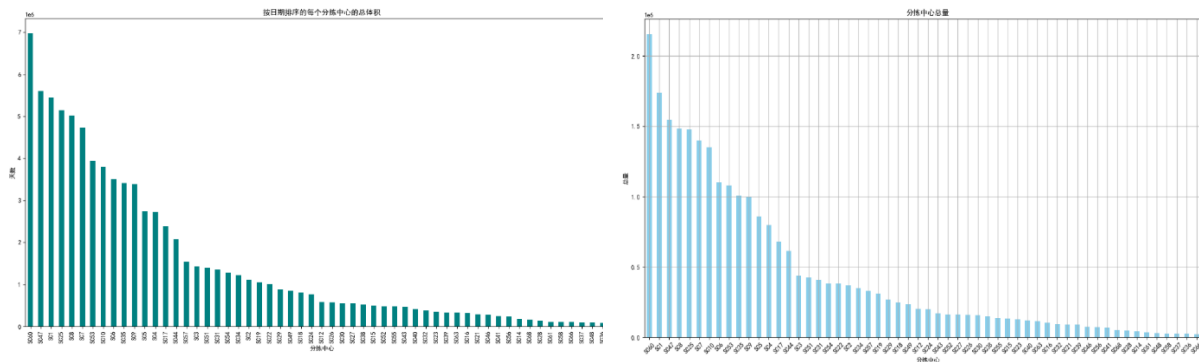


图 6：附件 1 与附件 2 分拣中心货量可视化

由图 6 可知附件 1 附件 2 中不同分拣中心的总货量比例是一致的，但是总货量数值存在差异。

2. 利用附件 2 中数据建立各个分拣中心不同小时的记录数小时—记录数之间的关系图：

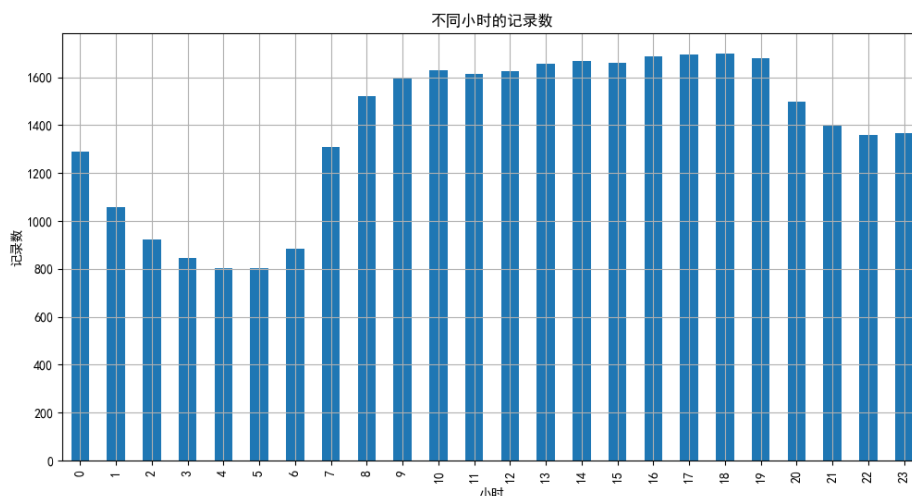


图 7：附件 2 数据可视化

由图 7 可以看出附件 2 中的不同分拣中心存在缺失值。

step3: 对附件 2 中的缺失值进行插值处理，本文采用线性插值进行处理。将插值

后的数据与附件 1 对比：

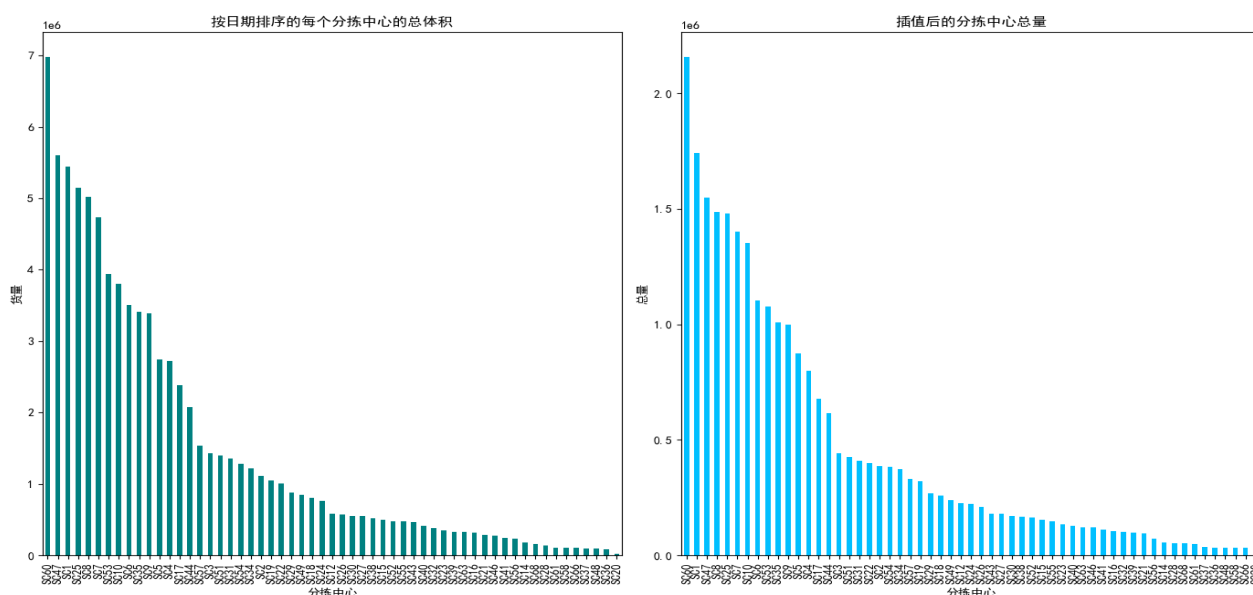


图 8：插值后分拣中心货量对比可视化

由图 8 可知附件 1 附件 2 中不同分拣中心总货量的总体趋势大致相同，但是具体总货量还是存在差异。这是因为附件 1 总日期数量为 122 而附件 2 总日期数量为 30，因此有此差距是正常的。

step4: 对不同日期每天的货量进行可视化：

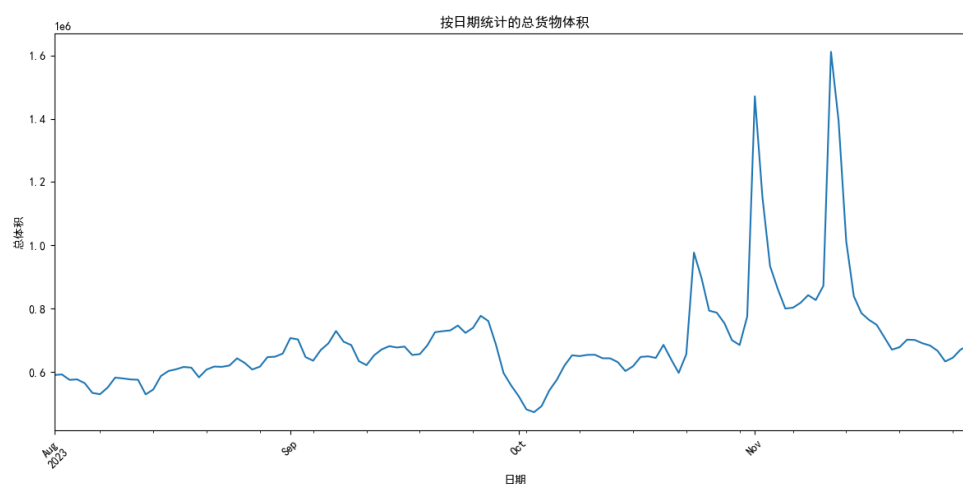


图 9：不同日期货量可视化

由图 9 可知货量随时间有所波动，特别的在 11 月份的波动较大，可能受到“双十一”活动的影响，群众的购买欲较大，所以 11 月份的总货量呈现出明显的上升趋势。

step5: 随机抽取五组分拣中心（SC5、SC51、SC24、SC7、SC29）的总货量的可视化：

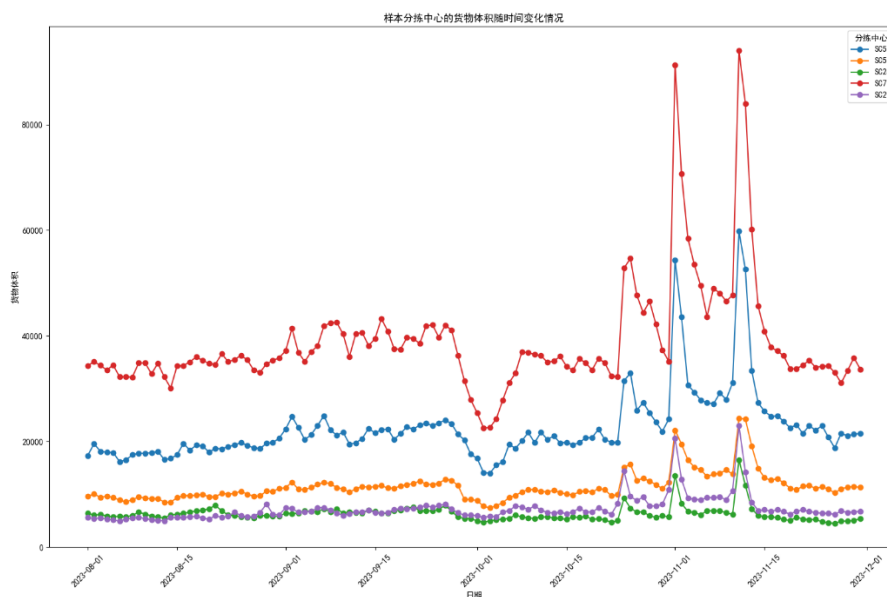


图 10：五个分拣中心每天货量可视化

由图 10 可知 SC60 分拣中心每天货量在整个时间轴都明显高于其余 4 个分拣中心，而 SC14 与 SC68 两个分拣中心每天的货量相对较低，可能是由于它们的地理位置不同使各个物流网络的运输线路间有所差异导致货物数量差异较明显。

step6: 由附件 1 过去 4 个月每天的货量数据，绘制出抽取的五个分拣中心的总货量的箱线图^[2]为：

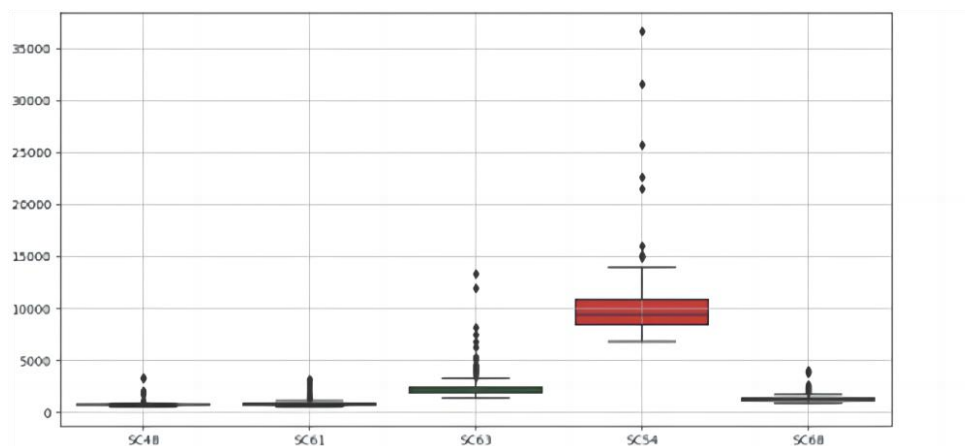


图 11：五个分拣中心每天货量的箱线图

由图 11（横坐标为分拣中心，纵坐标为每天货量）可以看出不同分拣中心之间的方差和均值存在较大差异。

step7: 对附件 1 中分拣中心 SC14 个月每天的货量进行可视化：



图 12: 4 个月每天货量可视化

由图 12（横坐标为日期，纵坐标为每日货量）从中可以看出该序列在 10 月 1 日、11 月分呈现出较大的波动，因此需要在此基础上构造平稳序列。

step8: ARIMA 时间序列预测模型的建立

1.差分阶数 d 的确定:任意选择一个分拣中心（SC12）通过 SPSS 软件得出 SC12 货量的 ADF 检验表^[3]为:

表 1: SC12 货量 ADF 检验表

变量	差分阶数	t	P	AIC	临界值		
					1%	5%	10%
预测货量	0	-0.553	0.881	-0.13	-3.77	-3.005	-2.643
	1	-17.221	0.000***	0.823	-3.689	-2.972	-2.625
	2	-2.593	0.095*	8.712	-3.788	-3.013	-2.646

注: ***、**、*分别代表 1%、5%、10%的显著性水平

该序列的 ADF 检验的结果显示，基于变量货量:在差分为 0 阶时，显著性 P 值为 0.881（大于 0.05），水平上不呈现显著性，不能拒绝原假设，该序列为不平稳的时间序列；在差分为 1 阶时，显著性 P 值为 0.000***（小于 0.05），水平上呈现显著性，拒绝原假设，该序列为平稳的时间序列，因此得到差分阶数 d 为 1。

2.自回归阶数 p 与偏自回归阶数 q 的确定:



图 13: 自相关图 (ACF)



图 14: 偏自相关图 (PACF)

基于 AIC 信息准则^[4]自动寻找最优参数，图几和图几可以确定自回归阶数 p 与偏自回归阶数 q 值都确定为 1。

通过上述分析确定建立 ARIMA (1,1,1) 模型。

step9: ARIM 时间序列预测模型的检验

1.通过 SPSS 软件对 ARIMA (1,1,1) 模型进行检验:

表 2: ARIMA 模型 (1,1,1) 检验表

项	符号	值
样本数量	Df Residuals	26
	N	30
	Q6(P 值)	10.096(0.001***)
Q 统计量	Q12(P 值)	13.655(0.034**)
	Q18(P 值)	16.354(0.176)
	Q24(P 值)	18.503(0.423)
信息准则	AIC	271.548
	BIC	277.017
拟合优度	R ²	0.992

注: ***, **, *分别代表 1%、5%、10%的显著性水平

对于货量这个变量，从 Q 统计量结果分析可以得到: Q18 在水平上不呈现显著性，不能拒绝模型的残差为白噪声序列的假设，同时模型的拟合优度 R^2 为 0.992，模型表现较为良好，模型基本满足要求。

2.通过 SPSS 软件绘制模型残差自相关图(ACF)与模型残差偏自相关图(PACF):

表: 模型残差自相关图(ACF)与模型残差偏自相关图(PACF)^[5]



图 15: 模型残差自相关图(ACF)



图 16: 模型残差自相关图(PACF)

由于时间序列要求模型残差为白噪声序列，由图几和图几可知相关系数与自相关系数大致都在虚线内，滑动平均模型(MA)残差为白噪声序列。

3.模型检验表为:

表 3: 模型检验表

	系数	标准差	t	P> t	0.025	0.975
常数	-2.12	7.168	-0.296	0.767	-16.169	11.928

ar.L1	0.906	0.286	3.172	0.002	0.346	1.466
ma.L1	0.974	1.415	0.688	0.491	-1.8	3.748
sigma2	399.929	549.214	0.728	0.467	-676.512	1476.369

注：***、**、*分别代表 1%、5%、10%的显著性水平

通过分析表 3，对于变量货量基于 AIC 信息准则自动寻找最优参数，模型结果为 ARIMA 模型 (1,1,1) 检验表，模型公式如下：

$$y(t) = -2.12 + 0.906 * y(t-1) + 0.974 * \varepsilon(t-1) \quad (1)$$

step10: 绘制出时间序列图检验图并得出 SC12 未来 30 天每天货量预测结果表（见附录）：



图 17：时间序列检验图

由图 17 可知该模型预测值与真实值基本吻合，预测效果良好。

表 4：SC12 分拣中心未来 10 天货量预测结果表

分拣中心	日期	货量	分拣中心	日期	货量
SC12	2023/12/1	4049	SC12	2023/12/6	4164
SC12	2023/12/2	3933	SC12	2023/12/7	4210
SC12	2023/12/3	3954	SC12	2023/12/8	4253
SC12	2023/12/4	4032	SC12	2023/12/9	4298
SC12	2023/12/5	4108	SC12	2023/12/10	4344

表 4 是对 SC12 未来 10 天预测结果的展示，剩余预测结果见结果表 1 并绘制出各分拣中心未来 30 天货量预测图（见支撑材料）。

7.1.3 对未来 30 天每小时货量的预测

将日频预测转化为时频预测：

由于时频间隔较短，大多数传统的时间序列模型无法捕捉短频时间的相关性，所以本文通过日频的预测值推算出对应时间段的货量预测值。

step1: 某个分拣中心 11 月份某天某时刻的货量 m 与整天的总货量 M 相除得出一个小时级别的比率 $r_{i,j}$ ：

$$r_{i,j} = \frac{m}{M} \quad (2)$$

step2: 将得出的比率 $r_{i,j}$ 与结果 1 中的数据相乘得出未来 30 天每天每小时的货量。

7.1.4 对未来 30 天每小时货量的预测结果求解

step1: 通过 Python 编程得出各分拣中心每天每小时级别比率结果:

表 5: SC32 分拣中心 11 月 4 日 5 个时间点的货量比率

日期时间	分拣中心	货量	时刻货量	日期	货量	每日总货量	货量比率
2023-11-04 01:00:00	SC32	0		2023-11-04	3937		0
2023-11-04 02:00:00	SC32	0		2023-11-04	3937		0
2023-11-04 13:00:00	SC32	16		2023-11-04	3937		0.004064008
2023-11-04 03:00:00	SC32	40		2023-11-04	3937		0.01016002
2023-11-04 17:00:00	SC32	55		2023-11-04	3937		0.013970028

表 5 展示了 SC32 分拣中心 11 月 4 日 5 个时间点的货量比率, 其余货量比率结果见支撑材料。

step2: 各分拣中心未来 30 天每小时的货量预测结果:

表 6: SC1 分拣中心 12 月 1 日 0-4 点货量预测表

日期	时间	分拣中心	每小时预测货量	日期	时间	分拣中心	每小时预测货量
2023/12/1	0	SC1	2970	2023/12/1	5	SC1	2135
2023/12/1	1	SC1	2988	2023/12/1	6	SC1	257
2023/12/1	2	SC1	2895	2023/12/1	7	SC1	1170
2023/12/1	3	SC1	2917	2023/12/1	8	SC1	1893
2023/12/1	4	SC1	2751	2023/12/1	9	SC1	1585

表 6 展示了 SC1 分拣中心 12 月 1 日 0-4 点货量预测结果, 剩余预测结果见结果表 2。

7.2 问题二的模型的建立与求解

7.2.1 对路线改变后各个分拣中心未来 30 天每天及每小时货量预测

step1: 为了找出变化的路线对应的总货量, 通过 Python 编程对附件 3 与附件 4 中的路线进行可视化。

step2: 将附件三中的数据 and 附件四中的数据进行匹配 看哪些运输线路发生了改变, 对于每一条变化的线路, 需要计算出它的总货量。

假设变化的路线为 $L_{i,j}$, 其平均货量为 $Y_{i,j}$, 则变化路线的总货量为:

$$Y = \sum_{j=1}^n L_{i,j} \quad (3)$$

其中 n 为变化路线的数量, $Y_{i,j}$ 为路线变化路线 $L_{i,k}$ 从分拣中心 j 出发的平均货量。

step3: 通过 Python 编程绘制出当附件 4 相对于附件 3 中分拣中心运输路线发生变化时, 每条路线上到达其他分拣中心的路线的占比情况。

step4: 依据上述分析得出的路线占比情况进一步计算出货量变化率并随之对之前对各个分拣中心未来 30 天每天及每小时的货量预测值进行修改。

7.2.2 路线改变后各分拣中心未来 30 天每天及每小时货量预测结果求解

step1: 附件 3 与附件 4 线路的可视化结果为:

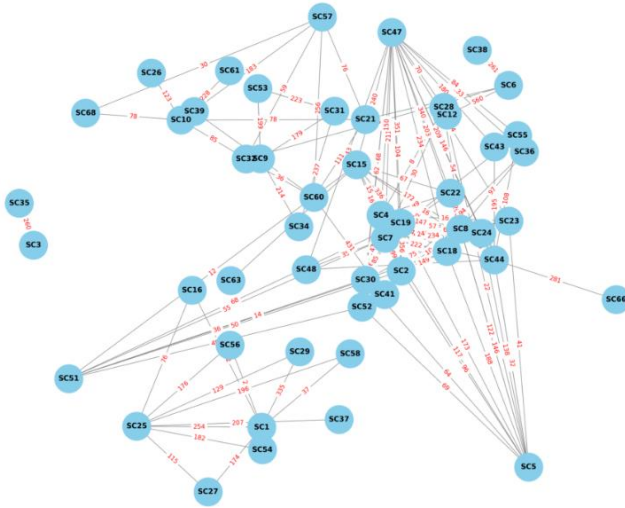


图 18：附件 3 路线以及路线之间货量可视化

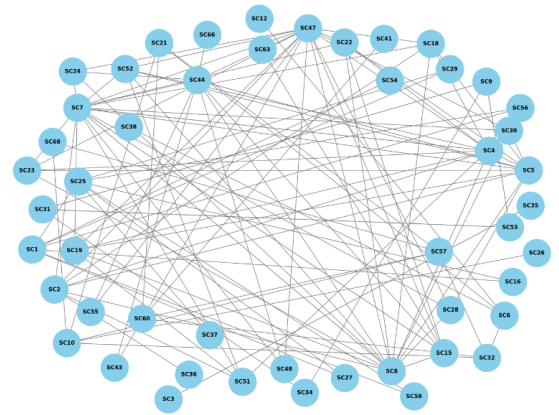


图 19：附件 4 分拣中心之间路线可视化

通过前文对附件 3 与附件 4 路线的可视化，提取出在附件 3 中存在而附件 4 中不存在的路线：

表 7：被移除的路线

始发分拣中心	到达分拣中心	始发分拣中心	到达分拣中心
SC1	SC25	SC36	SC47
SC18	SC51	SC36	SC8
SC19	SC15	SC39	SC60
SC2	SC19	SC4	SC15
SC24	SC5	SC51	SC15
SC28	SC4	SC54	SC25
SC55	SC7	SC61	SC10

在附件 4 中新增的路线为：

表 8：被更改的路线

始发分拣中心	到达分拣中心	始发分拣中心	到达分拣中心
SC31	SC9	SC5	SC4

step2: 对于上面列出的 14 条变化的路线，计算出它们在附件 3 的总货量分别为：

表 9：14 条变化路线上的总货量

始发分拣中心	到达分拣中心	总货量	始发分拣中心	到达分拣中心	总货量
SC1	SC25	254	SC36	SC47	97
SC18	SC51	14	SC36	SC8	119
SC19	SC15	336	SC39	SC60	15
SC2	SC19	356	SC4	SC15	12
SC24	SC5	138	SC51	SC15	182
SC28	SC4	8	SC54	SC25	128
SC55	SC7	133	SC61	SC10	228

step3: 通过 Python 编程绘制出当附件 4 中分拣中心运输路线发生变化时，每条路线上到达其他分拣中心的路线的占比情况为：

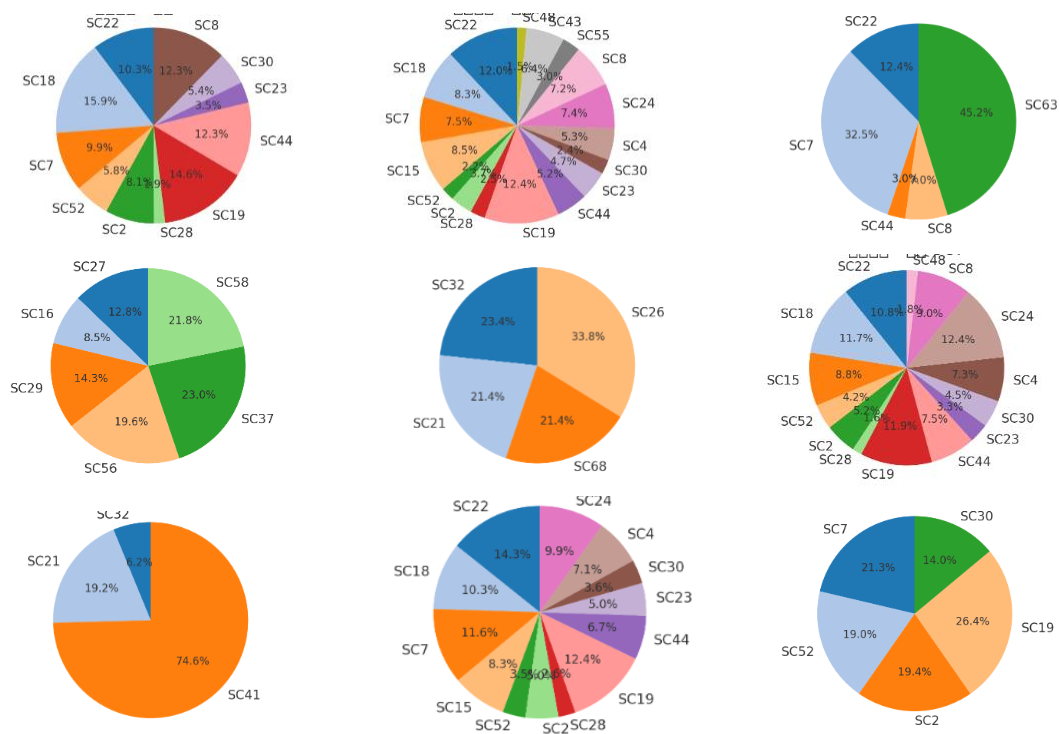


图 20：变化的运输路线到达其他分拣中心路线占比情况

由图几可知运输路线改变之后，分析第三个小图：SC63 分拣中心到 SC15 分拣中心路线的货量占到达 SC15 分拣中心总货量比值的 45.2%（最大占比），SC44 占比最小。其余分拣中心的占比情况同理可得。

step4: 计算货量变化率并修改先前预测值

1. 首先得出修正后的货量数据:

需要将附件 3 中移除路线各自的总货量 Y 按照相应比例分配到以该路线对应的到达分拣中心的相应路线上。令分配比例为 q ，则分配后的货量为 $Y_{i,j} = Y * q$ 。在该公式的基础上通过 Python 编程对修正后的货量数据进行可视化:

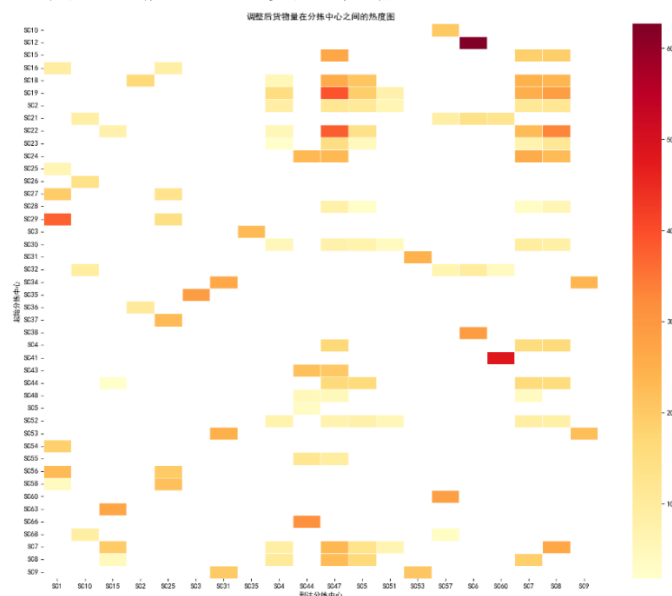


图 21：各个分拣中心调整后货量热力图

热力图中每个单元格表示对应的起始分拣中心到达分拣中心的货量。颜色越深代表货量越多，颜色越浅代表货量越少。由图 21 可知 SC12-SC6 该路线的货量最高；而 SC44-SC15 与 SC23-SC4 路线的货量最低。

2.将路线改变前后分拣中心的总货量进行对比：



图 22：分拣中心货量改变前后对比柱状图

由图 22 可知 SC5、SC47、SC7、SC8 分拣中心与其他分拣中心的货量流通较广，其所处地理位置可能较优；而 SC60 分拣中心与其他分拣中心的货量流通较少，其所处地理位置可能较偏僻。

2.然后计算变化率：

为了计算变化率，需要依据历史货量与重新分配后的货量。令过去各个分拣中心平均货量为 $Y_{p,j}$ ，则货量的变化率可表示为：

$$r_j = \frac{Y_{n,j}}{Y_{p,j}} - 1 \quad (4)$$

其中 $Y_{n,j}$ 为重新分配后的总货量， $Y_{p,j}$ 表示历史分拣中心 j 平均货量。通过上述公式利用 Python 编程得出各分拣中心的货量变化率图：

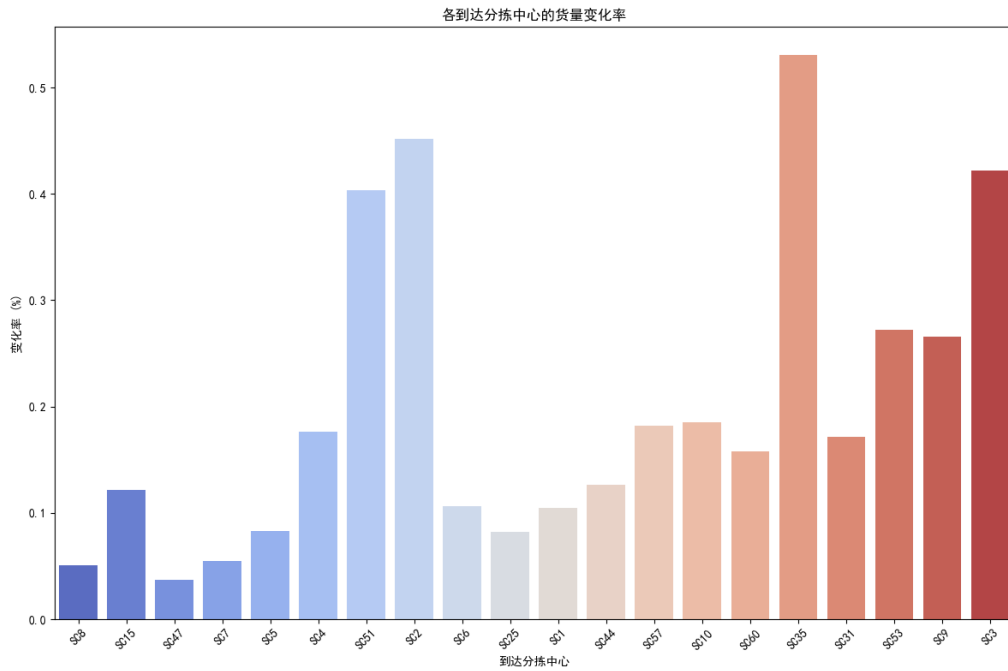


图 23: 各分拣中心货量变化率

由图 23 可知 SC35 的货量变化率最高，SC47 分拣中心的变化率最低。

3.最后通过计算出的变化率修正先前的预测结果：

令初始预测结果为 $P_{p,j}$ ，则修改后的预测结果为 $P_{n,j}$ ，公式为：

$$P_{p,j} = P_{n,j} * (1 + r_j) \quad (5)$$

通过上述公式利用 Python 编程得出修改之后未来 30 天每天及每小时的预测结果如表 10 所示。（由于篇幅有限，这里仅展示 SC1 的 12 月 1 日每小时结果，其余结果见结果表 2）

表 10：SC1 分拣中心 12 月 1 日 24 个小时货量预测结果图

日期	时间	分拣中心	预测货量	日期	时间	分拣中心	预测货量
2023/12/1	0	SC1	2970	2023/12/1	12	SC1	2298
2023/12/1	1	SC1	2988	2023/12/1	13	SC1	2129
2023/12/1	2	SC1	2895	2023/12/1	14	SC1	304
2023/12/1	3	SC1	2917	2023/12/1	15	SC1	496
2023/12/1	4	SC1	2751	2023/12/1	16	SC1	918
2023/12/1	5	SC1	2135	2023/12/1	17	SC1	942
2023/12/1	6	SC1	257	2023/12/1	18	SC1	1109
2023/12/1	7	SC1	1170	2023/12/1	19	SC1	1846
2023/12/1	8	SC1	1893	2023/12/1	20	SC1	2591
2023/12/1	9	SC1	1585	2023/12/1	21	SC1	2586
2023/12/1	10	SC1	1424	2023/12/1	22	SC1	2857
2023/12/1	11	SC1	1707	2023/12/1	23	SC1	2933

7.3 问题三的模型的建立与求解

7.3.1 基于纯整数规划模型探究未来 30 天每个分拣中心每个班次出勤人数

已知每个分拣中心每天有 6 个班次，令各个班次的正式工为 $x_i (i = 1, 2, 3 \cdots 60)$ ，各个班次临时工为 $y_i (i = 1, 2, 3 \cdots 60)$ ，当正式工不够用时需要雇佣临时工。

其中所有班次的所有正式工人数为：

$$X = \sum_{i=1}^6 x_i \quad (6)$$

所有班次所有临时工人数为：

$$Y = \sum_{i=1}^6 y_i \quad (7)$$

设 Q_h 为第 h 小时的预测货量， I 为所有班次的集合，每个班次 $i \in I$ ，则每个班次开始时间到结束时间表示为 (h_{start}, h_{end}) ， N_i 为班次 i 中的预测货量。

先对时间做归一化处理，将24个小时分为24个单独的时段，将预测货量分配到这些时间段内，每个班次都是通过其工作开始时间 h_{start} 与工作结束时间 h_{end} 来定义的。这些工作开始时间与工作结束时间都是整点。

为了在给定时间段内，依据预测货量选择合适的班次并更新该班次的货量以适应实际需求的过程，对于每小时 h 需要进行以下操作：

1. 找出可以处理 h 个小时预测货量的班次集合 $A_h = \{i \in I | h_{start} \leq h < h_{end}\}$ ，然后在集合 A_h 中找到预测货量最少的班次 i^* ，定义 $i^* = \min N_i$ （集合 A_h 包含了所有时间段 h 工作的班次）；

2. 将第 h 个小时的预测货量加到 N_{i^*} 上，更新 $N_{i^*} = N_{i^*} + Q_h$
将这些预测货量分配给这些班次让每个班次工作量尽可能相同。

step1: 参数定义：

表 11: 参数定义表

参数	定义	参数	定义
D	天数（本题为 30 天）	η_x	正式工工作效率（本题为 25 包裹/小时）
I	每天的班次数（本题为 6 个班次）	η_y	临时工工作效率（本题为 20 包裹/小时）
X	正式工人数（本题为 60 人）	$\eta_{x,y}$	实际工作效率
Y	临时工人数	$L_{k,d,i}$	第 k 个分拣中心第 d 天第 i 个班次货量需求
K	分拣中心的数量（本题为 57 个）	$X_{k,d,i}$	第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数
$Y_{k,d,i}$	第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数		

step2: 决策变量：

1. $X_{k,d,i}$: 第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数；
2. $Y_{k,d,i}$: 第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数。

step3: 目标函数：

题目要求在每天货量处理完成的基础上，安排的人天数尽可能小，目标函数为：

$$\min z = \sum_{d=1}^D \sum_{k=1}^K \sum_{i=1}^I (X_{k,d,i} + Y_{k,d,i}) \quad (8)$$

其中 z 为安排的人天数， D 为需要预测货量的总天数（30 天）， d 为需要预测货量的某一天； K 为分拣中心的总数量（共 57 个分拣中心）， k 某个具体的分拣中心； I 为一天的班次数（6 个班次）； $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； $Y_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数。

step4: 约束条件:

1. 确保每个分拣中心每个班次货量满足基本货量需求:

$$X_{k,d,i} * \eta_x + Y_{k,d,i} * \eta_y \geq L_{k,d,i} (\forall d \in \{1, 2, 3 \dots D\}, \forall i \in \{1, 2, 3 \dots I\}) \quad (9)$$

其中 $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； η_x 正式工工作效率（本题为 25 包裹/小时）； $Y_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数； η_y 为临时工工作效率（本题为 20 包裹/小时）。

2. 确保每个分拣中心每天的正式工人员数不超过总人数:

$$\sum_{i=1}^I X_{k,d,i} \leq X (\forall d \in \{1, 2, 3 \dots D\}) \quad (10)$$

其中 $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； X 为最高正式工人数（60 人）。

3. 优先使用正式工人力资源，同时确保每天所有正式工都能安排到班次工作:

$$\sum_{i=1}^I Y_{k,d,i} + X_{k,d,i} \leq X (\forall d \in \{1, 2, 3 \dots D\}) \quad (11)$$

其中 $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； $Y_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数； X 为最高正式工人数（60 人）。

正式工与临时工实际效率为:

$$\eta_{x,y} = \frac{X_{k,d,i}}{X_{k,d,i} + Y_{k,d,i}} * \eta_x + \frac{Y_{k,d,i}}{X_{k,d,i} + Y_{k,d,i}} * \eta_y \quad (12)$$

其中 $\eta_{x,y}$ 实际工作效率，其中 $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； $Y_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数； η_x 正式工工作效率（本题为 25 包裹/小时）； η_y 为临时工工作效率（本题为 20 包裹/小时）。

4. 确保每个分拣中心每天工作实际小时人效尽量均衡:

$$\left| \frac{\sum_{k=1}^K (X_{k,(d+1),i} + Y_{k,(d+1),i}) * \eta_{x,y}}{8} - \frac{\sum_{k=1}^K (X_{k,d,i} + Y_{k,d,i}) * \eta_{x,y}}{8} \right| \leq \beta \quad (13)$$

其中 K 为分拣中心的总数量（共 57 个分拣中心）；其中 $X_{k,(d+1),i}$ 与 $X_{k,d,i}$ 为第 k 个分拣中心第 $(d+1)$ 天与第 d 天第 i 个班次分配的正式工人数； $Y_{k,(d+1),i}$ 与 $Y_{k,d,i}$ 为第 k 个分拣中心第 $(d+1)$ 天与第 d 天第 i 个班次分配的临时工人数； $\eta_{x,y}$ 为实际工作效率； β 为最大差值； β 越小人效越均衡。

5. 正式工人数与临时工人数大于 0:

$$\begin{cases} X_{k,d,i} \geq 0 \\ Y_{k,d,i} \geq 0 \end{cases} \quad (14)$$

其中 $\eta_{x,y}$ 实际工作效率，其中 $X_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的正式工人数； $Y_{k,d,i}$ 为第 k 个分拣中心第 d 天第 i 个班次分配的临时工人数。

综上所述：纯整数线性规划模型汇总如下：

$$\min z = \sum_{d=1}^D \sum_{k=1}^K \sum_{i=1}^I (X_{k,d,i} + Y_{k,d,i}) \quad (15)$$

$$s.t. \left\{ \begin{array}{l} X_{k,d,i} * \eta_x + Y_{k,d,i} * \eta_y \geq L_{k,d,i} (\forall d \in \{1, 2, 3 \cdots D\}, \forall i \in \{1, 2, 3 \cdots I\}) \\ \sum_{i=1}^I X_{k,d,i} \leq X (\forall d \in \{1, 2, 3 \cdots D\}) \\ \sum_{i=1}^I Y_{k,d,i} + X_{k,d,i} \leq X (\forall d \in \{1, 2, 3 \cdots D\}) \\ \left| \frac{\sum_{k=1}^K (X_{k,(d+1),i} + Y_{k,(d+1),i}) * \eta_{x,y}}{8} - \frac{\sum_{k=1}^K (X_{k,d,i} + Y_{k,d,i}) * \eta_{x,y}}{8} \right| \leq \beta \\ X_{k,d,i} \geq 0 \\ Y_{k,d,i} \geq 0 \end{array} \right. \quad (16)$$

7.3.2 模型的求解

通过 Python 编程得出结合上述纯整数规划模型得出未来 30 天每个分拣中心每个班次出勤人数为：

表 12：SC1 分拣中心 5 个班次的出勤情况

分拣中心	日期	班次	正式工	临时工人数	分拣中心	日期	班次	正式工	临时工
SC1	2023/12/1	00:00-08:00	26	0	SC1	2023/12/1	16:00-24:00	0	26
SC1	2023/12/1	05:00-13:00	26	0	SC1	2023/12/10	00:00-08:00	28	0
SC1	2023/12/1	08:00-16:00	8	18	SC1	2023/12/10	05:00-13:00	28	0
SC1	2023/12/1	12:00-20:00	0	26	SC1	2023/12/10	08:00-16:00	4	24
SC1	2023/12/1	14:00-22:00	0	26	SC1	2023/12/10	12:00-20:00	0	28

表 12 为 SC1 分拣中心 12 月 1 日 5 个班次的出勤情况，其余分拣中心的出勤情况见结果表 5。

7.4 问题四的模型的建立与求解

7.4.1 基于纯整数线性规划模型探究未来 30 天每名正式工与临时工的班次出勤计划

step1: 数据筛选：

题目中以 SC60 为例探究未来 30 天每名正式工与临时工的出勤计划，需要先通过 EXCEL 从附件 1 与附件 2 中筛选出分拣中心 SC60 分拣中心未来 30 天及每小时货量预测结果。

step1: 参数定义：

表 13：参数定义表

参数	定义	参数	定义
D	天数（本题为 30）	η_x	正式工工作效率（本题为 25 包裹/小时）
I	每天班次数（本题为 6 个班次）	η_y	临时工工作效率（问题为 20 包裹/小时）
X	正式工人数（本题为 200 人）	$\eta_{x,y}$	实际工作效率
Y	临时工人数	$L_{j,d,i}$	第 j 个分拣中心第 d 天第 i 个班次的货量需求
J	第 j 名员工		

其中实际工作效率 $\eta_{x,y} = \frac{X_{k,d,i}}{X_{k,d,i} + Y_{k,d,i}} * \eta_x + \frac{Y_{k,d,i}}{X_{k,d,i} + Y_{k,d,i}} * \eta_y$

step2: 决策变量:

引入 0-1 变量, 设 $X_{j,d,i}$ 为二进制变量, 代表第 j 名正式工第 d 天第 i 个班次出勤情况:

$$X_{j,d,i} = \begin{cases} 1 \\ 0 \end{cases} \quad (17)$$

其中当 $X_{j,d,i}$ 为 1 时, 表明第 j 名正式工在第 d 天第 i 个班次出勤; 当 $X_{j,d,i}$ 为 0 时, 表明第 j 名正式工在第 d 天第 i 个班次未出勤。

假设第 d 天第 i 个班次雇佣的临时工人数为 $Y_{d,i}$ 个临时工, 则应满足条件: $Y_{d,i} \in N$

step3: 目标函数:

要求安排的人天数尽可能少, 即保证货量完成的基础上正式工与临时工的人数尽可能少:

令正式工的人数为 α_1 , 则有:

$$\alpha_1 = \sum_{j=1}^J \sum_{d=1}^D \sum_{i=1}^I X_{j,d,i} \quad (18)$$

其中 J 为第 j 名正式工; D 为预测的总天数 (共 30 天); I 为每天的班次数 (共 6 个班次); $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况。

雇佣的临时工人数为 α_2 , 则有:

$$\alpha_2 = \sum_{d=1}^D \sum_{j=1}^J Y_{d,i} \quad (19)$$

其中 J 为第 j 名临时工; D 为预测的总天数 (共 30 天); $Y_{d,i}$ 为第 d 天第 i 个班次的临时工人数。

将上式相加求最小值作为总目标函数为:

$$\min \alpha = \alpha_1 + \alpha_2 \quad (20)$$

step4: 约束条件:

1. 出勤率的约束: 每名正式工的出勤率 (出勤天数/总天数) 不能高于 85%:

$$\frac{\sum_{j=1}^J \sum_{d=1}^D \sum_{i=1}^I X_{j,d,i}}{30} \leq 85\% \quad (21)$$

其中 J 为第 j 名正式工； D 为预测的总天数（共 30 天）； $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况。

2.连续出勤约束与货量处理的约束：确保每个班次的货量处理完成：

$$X_{j,d,i} * \eta_x + Y_{j,d,i} * \eta_y \geq L_{j,d,i} (\forall d \in \{1, 2, 3 \cdots D\}, \forall i \in \{1, 2, 3 \cdots I\}) \quad (22)$$

其中 $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况； η_x 为正式工工作效率（本题为 25 包裹/小时）； $Y_{j,d,i}$ 为第 j 名临时工在第 d 天第 i 个班次出勤人数； η_y 为临时工工作效率（本题为 20 包裹/小时）； $L_{j,d,i}$ 为第 j 个分拣中心第 d 天第 i 个班次的货量需求。

3.正式工出勤天数：不能超过 7 天：

$$\sum_{d=t}^{t+7} X_{j,d,i} \leq 7 \quad (23)$$

其中 $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况。

4. 确保每个分拣中心每天工作实际小时人效尽量均衡：

$$\left| \frac{\sum_{k=1}^K (X_{k,(d+1),i} + Y_{k,(d+1),i}) * \eta_{x,y}}{8} - \frac{\sum_{k=1}^K (X_{k,d,i} + Y_{k,d,i}) * \eta_{x,y}}{8} \right| \leq \beta \quad (24)$$

其中 K 为分拣中心的总数量（共 57 个分拣中心）；其中 $X_{k,(d+1),i}$ 与 $X_{k,d,i}$ 为第 k 个分拣中心第 $(d+1)$ 天与第 d 天第 i 个班次分配的正式工人数； $Y_{k,(d+1),i}$ 与 $Y_{k,d,i}$ 为第 k 个分拣中心第 $(d+1)$ 天与第 d 天第 i 个班次分配的临时工人数； $\eta_{x,y}$ 为实际工作效率； β 为最大差值： β 越小人效越均衡。

5.每天出勤约束：正式工出勤率尽量平衡，必须满足各个员工至少在 6 个班次中出勤 1 次，使用二进制决策变量进行约束：

$$\sum_{i=1}^I X_{j,d,i} = 1 \quad (25)$$

其中其中 $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况。

6. 正式工人数与临时工人数大于 0：

$$\begin{cases} X_{j,d,i} \geq 0 \\ Y_{j,d,i} \geq 0 \end{cases} \quad (26)$$

其中其中 $X_{j,d,i}$ 为第 j 名正式工在第 d 天第 i 个班次出勤状况； $Y_{j,d,i}$ 为第 j 名临时工在第 d 天第 i 个班次出勤人数。

综上所述，建立如下纯整数规划模型：

$$\min \alpha = \min(\alpha_1 + \alpha_2) = \sum_{j=1}^J \sum_{d=1}^D \sum_{i=1}^I X_{j,d,i} + \sum_{d=1}^D \sum_{i=1}^I Y_{d,i} \quad (27)$$

$$s.t. \left\{ \begin{array}{l} \frac{\sum_{j=1}^J \sum_{d=1}^D \sum_{i=1}^I X_{j,d,i}}{30} \leq 85\% \\ X_{j,d,i} * \eta_x + Y_{j,d,i} * \eta_y \geq L_{j,d,i} (\forall d \in \{1, 2, 3 \cdots D\}, \forall i \in \{1, 2, 3 \cdots I\}) \\ \sum_{d=t}^{t+7} X_{j,d,i} \leq 7 \\ \left| \frac{\sum_{k=1}^K (X_{k,(d+1),i} + Y_{k,(d+1),i}) * \eta_{x,y}}{8} - \frac{\sum_{k=1}^K (X_{k,d,i} + Y_{k,d,i}) * \eta_{x,y}}{8} \right| \leq \beta \\ \sum_{i=1}^I X_{j,d,i} = 1 \\ X_{j,d,i} \geq 0 \\ Y_{j,d,i} \geq 0 \end{array} \right. \quad (28)$$

7.4.2 模型的求解

通过 Python 编程利用上述构建的纯整数规划模型得出未来 30 天每名正式工与临时工的出勤计划为：

表 14：SC60 分拣中心 12 月 1 日 16:00 到 24:0030 天出勤计划一部分展示

分拣中心	日期	班次	出勤员工	分拣中心	日期	班次	出勤员工
SC60	2023/12/1	16:00-24:00	正式工 1	SC60	2023/12/1	16:00-24:00	正式工 6
SC60	2023/12/1	16:00-24:00	正式工 2	SC60	2023/12/1	16:00-24:00	正式工 7
SC60	2023/12/1	16:00-24:00	正式工 3	SC60	2023/12/1	16:00-24:00	正式工 8
SC60	2023/12/1	16:00-24:00	正式工 4	SC60	2023/12/1	16:00-24:00	正式工 9
SC60	2023/12/1	16:00-24:00	正式工 5	SC60	2023/12/1	16:00-24:00	正式工 10

表 14 展示的是 SC60 分拣中心 12 月 1 日 16:00 到 24:0030 天出勤计划的一部分，其余出勤计划结果见结果表 6。

八、模型的评价

8.1 模型的优点

1. 本文采用 ARIMA 时间序列预测模型对 56 个分拣中心未来 30 天每天货量进行预测，能有效处理具有线性特征的时间序列。

2. 本文对不平稳的时间序列进行差分使之变得更加平稳，使得 ARIMA 时间序列预测模型预测结果更加准确。

3. 本文使用纯整数规划模型制定出勤计划，可以帮助分拣中心合理分配有效资源，实现最大化利益。

8.2 模型的缺点

1. ARIMA 时间序列预测模型对于存在季节性、趋势变化等复杂特征的数据，可能效果不佳。

2. 本文在使用纯整数规划模型制定出勤计划时未能考虑到所有相关约束条件，结果优化空间很大。

8.3 模型的改进

1.使用 ARIMA 时间序列预测模型的过程中存在季节性因素时，可以引入季节性差分或引入季节性模型（如季节性自回归移动平均模型）。

2.使用纯整数规划模型解决问题时，可以与随即规划模型相结合，考虑不确定因素对结果的影响。

8.4 模型的推广

1. ARIMA 时间序列预测模型可以推广到金融市场用于股票价格、汇率等的预测。

2.纯整数规划模型还可推广到生产调度确定生产过程中各种任务的顺序与时间安排、库存管理中控制库存水平确保供应与需求的平衡等方面。

九、参考文献

- [1]殷志祥,许进.分子信标芯片计算在 0-1 整数规划问题中的应用[J].生物数学学报, 2007, 22(3):6.DOI:10.3969/j.issn.1001-9626.2007.03.025.
- [2]朱兵.箱线图及其在 JCR 网络版中的应用[J].农业图书情报学刊, 2011(08):15-18.DOI:10.3969/j.issn.1002-1248.2011.08.004.
- [3]陈昭.时序非平稳性 ADF 检验法的理论与应用[J].广州大学学报: 自然科学版, 2008, 7(5):6.DOI:10.3969/j.issn.1671-4229.2008.05.002.
- [4]沈新宁,王小龙,杜建洪.基于颜色自相关图和互信息的图像检索算法[J].计算机工程, 2014.DOI:CNKI:SUN:JSJC.0.2014-02-056.
- [5]张济民,王开文,池茂儒.基于 AIC 信息准则法的摆式列车倾摆伺服系统建模研究[J].中国铁道科学, 2003, 24(6):4.DOI:10.3321/j.issn:1001-4632.2003.06.002.

Python 运行代码（成功运行）

问题一：

附录 1

介绍：附件 1 缺失值的查找

```
import pandas as pd
import matplotlib.pyplot as plt
file_path = 'C:/用户/桌面/附件1.csv'
data = pd.read_excel(file_path)
sorting_days = data.groupby('分拣中心')['日期'].nunique()
sorted_sorting_days = sorting_days.sort_values(ascending=False)
plt.figure(figsize=(14, 8))
sorted_sorting_days.plot(kind='bar', color='skyblue')
plt.title('Number of Sorting Days per Sorting Center')
plt.xlabel('Sorting Center')
plt.ylabel('Number of Days')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

附录 2

介绍：附件 2 缺失值的查找

小时-记录数代码：

```
import pandas
as pd
import matplotlib.pyplot as plt
# 加载数据
file_path_2_updated = r"C:\Users\20157\Desktop\附件2.xlsx"
data_2_updated = pd.read_excel(file_path_2_updated)
# 计算每个分拣中心的总货量
total_volume_by_center_updated = data_2_updated.groupby('分拣中心')['货量'].sum().sort_values(ascending=False)
# 绘制条形图
plt.figure(figsize=(14, 8))
total_volume_by_center_updated.plot(kind='bar', color='skyblue')
plt.title('Total Volume by Sorting Center (Updated)')
plt.xlabel('Sorting Center')
plt.ylabel('Total Volume')
plt.grid(True)
plt.xticks(rotation=45) # Rotate labels to improve readability
plt.tight_layout() # Automatically adjust subplot parameters to give
specified padding
plt.show()
```

附录 3

介绍：附件 1 每个分拣中心过去 4 个月每天货量可视化

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
file_path = r'C:\Users\20157\Desktop\附件1.xlsx'
data = pd.read_excel(file_path)
sorting_days = data.groupby('分拣中心')['日期'].nunique()
sorted_sorting_days = sorting_days.sort_values(ascending=False)
```

```
plt.figure(figsize=(14, 8))
sorted_sorting_days.plot(kind='bar', color='skyblue')
plt.title('每个分拣中心的分拣天数')
plt.xlabel('分拣中心')
plt.ylabel('天数')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

附录 4

介绍：不同分拣中心的总货量可视化

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
file_path = r'C:\Users\20157\Desktop\附件1.xlsx'
data = pd.read_excel(file_path)
data_sorted = data.sort_values(by='日期')
total_volume = data_sorted.groupby('分拣中心')['货量']
total_volume.sum().sort_values(ascending=False)
plt.figure(figsize=(14, 8))
total_volume.plot(kind='bar', color='teal')
plt.title('按日期排序的每个分拣中心的总体积')
plt.xlabel('分拣中心')
plt.ylabel('天数')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

附录 5

介绍：不同日期的货量可视化

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
# 加载数据
file_path = r'C:\Users\20157\Desktop\附件1.xlsx'
data = pd.read_excel(file_path)

# 计算每个日期的总货量
daily_total_volume = data.groupby('日期')['货量'].sum()

# 绘制总货量随日期变化的图表
plt.figure(figsize=(12, 6))
daily_total_volume.plot(kind='line')
plt.title('按日期统计的总货物体积')
plt.xlabel('日期')
plt.ylabel('总体积')
plt.xticks(rotation=45) # Rotate labels for better readability
plt.tight_layout()
plt.show()
```

附录 6

介绍：五个分拣中心每天的货量可视化

```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
# 加载数据
file_path = r'C:\Users\20157\Desktop\附件1.xlsx'
data = pd.read_excel(file_path)

# 随机选取5个分拣中心
sampled_center_ids = data['分拣中心'].drop_duplicates().sample(n=5)

# 筛选出这些分拣中心的数据
sampled_center_data = data[data['分拣中心'].isin(sampled_center_ids)]

# 绘制时间序列图
plt.figure(figsize=(15, 10))
for center in sampled_center_ids:
    # 筛选每个中心的数据，并按日期排序
    center_data = sampled_center_data[sampled_center_data['分拣中心'] == center]
    center_data = center_data.sort_values('日期')
    plt.plot(center_data['日期'], center_data['货量'], marker='o',
label=center)

plt.title('样本分拣中心的货物体积随时间变化情况')
plt.xlabel('日期')
plt.ylabel('货物体积')
plt.legend(title='分拣中心')
plt.xticks(rotation=45) # 旋转日期标签以便阅读
plt.tight_layout() # 调整布局以防止重叠
plt.show()

```

附录 7

介绍：绘制五个分拣中心每天的货量的箱线图

```

import pandas as pd
import matplotlib.pyplot as plt
file_path = 'C:/用户/桌面/附件1.csv'
data = pd.read_excel(file_path)
sampled_center_ids = data['分拣中心'].drop_duplicates().sample(n=5)
sampled_center_data = data[data['分拣中心'].isin(sampled_center_ids)]
plt.figure(figsize=(15, 10))
for center in sampled_center_ids:
    center_data = sampled_center_data[sampled_center_data['分拣中心'] == center]
    center_data = center_data.sort_values('日期')
    plt.plot(center_data['日期'], center_data['货量'], marker='o',
label=center)
plt.title('Cargo Volume Over Time for Sampled Sorting Centers')
plt.xlabel('Date')
plt.ylabel('Cargo Volume')
plt.legend(title='Sorting Center')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

附录 8

介绍：对附件 2 进行线性插值后与附件 1 的对比图

```
import pandas as pd
import numpy as np
from statsmodels.tsa.arima.model import ARIMA
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
# 假设已经加载了数据为sortedData2, 一个包含多个DataFrame的列表
sortedData2 = [pd.read_csv(f"data{i}.csv") for i in range(1, 58)] # 根据实际情况调整文件名或加载方式
# 从DataFrame列表中提取第四列数据
data = np.array([df.iloc[:, 3].values for df in sortedData2]).T
# 初始化结果数组
result = np.zeros((720, 57))
# 估计和预测
for j in range(57):
    y = data[:, j]
    model = ARIMA(y, order=(24, 0, 0))
    fitted_model = model.fit()
    num_periods = 720
    forecast_result, conf_int = fitted_model.get_forecast(steps=num_periods).summary_frame(alpha=0.05).iloc[:, :2].T.values
    result[:, j] = forecast_result
# 替换负值为0
result[result < 0] = 0
# 绘制结果
plt.figure(figsize=(15, 10))
for i in range(30):
    plt.subplot(5, 6, i+1)
    plt.plot(data[:, i], label='Observed')
    plt.plot(np.arange(720, 1440), result[:, i], '.-', label='Forecast')
    plt.xlabel("Time")
    plt.ylabel("Case")
    plt.title("Forecast")
    plt.legend()
plt.tight_layout()
plt.show()
plt.figure(figsize=(15, 10))
for i in range(30, 57):
    plt.subplot(5, 6, i-30+1)
    plt.plot(data[:, i], label='Observed')
    plt.plot(np.arange(720, 1440), result[:, i], '.-', label='Forecast')
    plt.xlabel("Time")
    plt.ylabel("Case")
    plt.title("Forecast")
    plt.legend()
plt.tight_layout()
plt.show()
# 生成日期和小时数据
start_date = datetime(2023, 12, 1)
end_date = datetime(2023, 12, 30)
date_range = pd.date_range(start_date, end_date, freq='H')
# 展开日期和小时
expanded_date = np.repeat(date_range.date.astype(str), 57)
expanded_hour = np.tile(np.tile(np.arange(24), len(date_range.date)), 57)
# 整合结果
```

```
centers = np.concatenate([df.iloc[:720, 0].values for df in sortedData2])
result_flat = result.flatten()
# 保存结果到CSV
output_df = pd.DataFrame({
    '分拣中心': centers,
    '日期': expanded_date,
    '小时': expanded_hour,
    '货量': result_flat
})
```

附录 9

介绍：各分拣中心每天每小时级别比率

```
import pandas as pd
# 载入Excel文件
file1_path = r"C:\Users\20157\Desktop\附件1.xlsx"
file2_path = r"C:\Users\20157\Desktop\缺失值.xlsx"
# 读取文件数据
data_file1 = pd.read_excel(file1_path)
data_file2 = pd.read_excel(file2_path)
# 在data_file2中提取日期，并创建新的日期列
data_file2['日期'] = data_file2['日期时间'].dt.date
# 根据'分拣中心'和'日期'分组，计算每天每个中心的货量总和
daily_cargo_from_file2 = data_file2.groupby(['分拣中心', '日期']).agg({'货量': 'sum'}).reset_index()
# 将计算得到的每日货量总和与data_file1中的数据合并，以便拥有计算和提供的总货量数据
merged_data = pd.merge(data_file1, daily_cargo_from_file2, on=['分拣中心', '日期'], suffixes=('_时刻货量', '_每日总货量'))
# 计算每个时刻货量与当天总货量的比率
merged_data['货量比率'] = merged_data['货量_时刻货量'] / merged_data['货量_每日总货量']
# 将结果保存到新的Excel文件中
output_path = '分拣中心货量比率分析.xlsx'
merged_data.to_excel(output_path, index=False)
```

问题二：

附录 1

介绍：附件 3 与附件 4 各个分拣中心路线的可视化

```
import matplotlib.pyplot as plt
import networkx as nx
def plot_spaced_network(data, title, attachment_number):
    plt.figure(figsize=(25, 20))
    G = nx.DiGraph()
    for index, row in data.iterrows():
        G.add_node(row['始发分拣中心'])
        G.add_node(row['到达分拣中心'])
        if '货量' in data.columns:
            G.add_edge(row['始发分拣中心'], row['到达分拣中心'], weight=row['货量'])
        else:
            G.add_edge(row['始发分拣中心'], row['到达分拣中心'])
    pos = nx.spring_layout(G, k=3, iterations=50)
```

```

    nx.draw_networkx_nodes(G, pos, node_color='skyblue', node_size=3000)
    nx.draw_networkx_labels(G, pos, font_size=14, font_weight='bold')
    nx.draw_networkx_edges(G, pos, edge_color='gray', arrowstyle='->',
arrowsize=30)
    if '货量' in data.columns:
        edge_labels = nx.get_edge_attributes(G, 'weight')
        nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels,
font_color='red', font_size=12)
    plt.title(title)
    plt.axis('off')
plt.savefig(f"/mnt/data/attachment_{attachment_number}_spaced_2d_network.
png", bbox_inches='tight')
    plt.close()

```

附录 2

介绍：附件 3 与附件 4 被更改或移除的数据展现

```

import pandas as pd
import matplotlib.pyplot as plt
# 加载数据
file_path_2_updated = r"C:\Users\20157\Desktop\附件2.xlsx"
data_2_updated = pd.read_excel(file_path_2_updated)
# 计算每个分拣中心的总货量
total_volume_by_center_updated = data_2_updated.groupby('分拣中心')['货量
'].sum().sort_values(ascending=False)
# 绘制条形图
plt.figure(figsize=(14, 8))
total_volume_by_center_updated.plot(kind='bar', color='skyblue')
plt.title('Total Volume by Sorting Center (Updated)')
plt.xlabel('Sorting Center')
plt.ylabel('Total Volume')
plt.grid(True)
plt.xticks(rotation=45) # Rotate labels to improve readability
plt.tight_layout() # Automatically adjust subplot parameters to give
specified padding
plt.show()

```

附录 3

介绍：绘制分拣中心运输路线发生变化时到其他分拣中心路线占比饼状图

```

import pandas as pd
import matplotlib.pyplot as plt
# 加载修正后的货量数据
adjusted_cargo_data = pd.read_excel(r"C:\Users\20157\Desktop\附件4.csv")
# 指定的到达分拣中心列表
specified_arrival_centers = ['SC5', 'SC47', 'SC15', 'SC10', 'SC25',
'SC7', 'SC60', 'SC8', 'SC51']
# 绘制指定到达分拣中心的饼状图
plt.figure(figsize=(15, 10))
for i, center in enumerate(specified_arrival_centers, 1):
    # 提取每个指定到达分拣中心的货量数据
    center_data = adjusted_cargo_data[adjusted_cargo_data['到达分拣中心']
== center]
    if center_data['修正后货量'].sum() == 0:
        continue # 如果总货量为0, 跳过绘图
plt.figure(figsize=(15, 10))
# 定义一个鲜艳的颜色方案

```

```

colors = plt.cm.tab20.colors # 使用Matplotlib的'Tab20'配色方案, 它包括20种鲜艳的颜色
# 绘制饼状图
plt.subplot((len(specified_arrival_centers) + 2) // 3, 3, i)
plt.pie(center_data['修正后货量'], labels=center_data['始发分拣中心'],
autopct='%1.1f%%', startangle=90)
plt.axis('equal') # 确保饼图为圆形
plt.title(f'货量占比 - 到达 {center}')
plt.tight_layout()
plt.show()

```

附录 4

介绍: 路线货量前后对比条形图

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.pyplot as plt
# 设置字体
plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体作为默认字体
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
# 在绘图代码之前调用以上两行代码
# Load and prepare data
file_path = r"C:\Users\20157\Desktop\货量数据.xlsx" # Update this path
data = pd.read_excel(file_path)
specific_centers = ['SC5', 'SC47', 'SC15', 'SC10', 'SC25', 'SC7', 'SC60', 'SC8', 'SC51']
filtered_data = data[data['到达分拣中心'].isin(specific_centers)]
# Set up the matplotlib figure for 9 subplots
fig, axes = plt.subplots(3, 3, figsize=(18, 15))
axes = axes.flatten()
# Plot bars for each specific arrival center including total quantities
for i, center in enumerate(specific_centers):
    ax = axes[i]
    # Filter data for the specific arrival center
    center_data = filtered_data[filtered_data['到达分拣中心'] == center]
    # Calculate total quantities
    center_data['总货量'] = center_data['货量'] + center_data['新增货量']
    # Plot original and total quantities
    sns.barplot(x='始发分拣中心', y='货量', data=center_data, ax=ax,
color='blue', label='原始货量')
    sns.barplot(x='始发分拣中心', y='总货量', data=center_data, ax=ax,
color='red', alpha=0.6, label='总货量')
    ax.set_title(f'分拣中心 {center} 货量对比')
    ax.set_xlabel('始发分拣中心')
    ax.set_ylabel('货量')
    ax.legend()
# Adjust layout
plt.tight_layout()
plt.show()

```

附录 5

介绍: 修改路线后货量变化热力图

```

import pandas as pd
import matplotlib.pyplot as plt

```

```

import seaborn as sns

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 加载删除后的补全数据
adjusted_cargo_data = pd.read_excel(r"C:\Users\20157\Desktop\将删除的货量重新分配的值.xlsx")

# 创建透视表
pivot_table_adjusted = adjusted_cargo_data.pivot_table(index='始发分拣中心', columns='到达分拣中心', values='修正后货量', aggfunc='sum')

# 绘制热力图
plt.figure(figsize=(14, 12)) # 增加热力图尺寸
sns.heatmap(pivot_table_adjusted, cmap="YlOrRd", linewidths=.5, annot=False) # 将 annot 设置为 False
plt.title('调整后货物量在分拣中心之间的热度图')
plt.ylabel('起始分拣中心')
plt.xlabel('到达分拣中心')
plt.tight_layout()
plt.show()

```

附录 6

介绍：路线货量前后对比图

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 加载数据
data_past = pd.read_excel(r"C:\Users\20157\Desktop\3.xlsx")
data_future = pd.read_excel(r"C:\Users\20157\Desktop\4.xlsx")

# 确定过去和未来的运输路线
past_routes_set = set(zip(data_past['始发分拣中心'], data_past['到达分拣中心']))
future_routes_set = set(zip(data_future['始发分拣中心'], data_future['到达分拣中心']))

# 识别被删除和保留的路线
removed_routes = past_routes_set - future_routes_set
remaining_routes = past_routes_set & future_routes_set

# 计算删除路线的总货量
removed_routes_data = data_past[data_past.apply(lambda x: (x['始发分拣中心'], x['到达分拣中心']) in removed_routes, axis=1)]
total_removed_cargo = removed_routes_data['货量'].sum()

# 计算保留路线的货量，并分配删除的货量
remaining_routes_data = data_past[data_past.apply(lambda x: (x['始发分拣中心'], x['到达分拣中心']) in remaining_routes, axis=1)]
total_remaining_cargo = remaining_routes_data['货量'].sum()
remaining_routes_data['占比'] = remaining_routes_data['货量'] / total_remaining_cargo
remaining_routes_data['新增货量'] = remaining_routes_data['占比'] * total_removed_cargo
remaining_routes_data['修正后货量'] = remaining_routes_data['货量'] + remaining_routes_data['新增货量']

# 准备绘图数据

```

```

comparison_data = remaining_routes_data[['始发分拣中心', '到达分拣中心', '货量', '修正后货量']].copy()
comparison_data['货量类型'] = '原始货量'
adjusted_data = comparison_data[['始发分拣中心', '到达分拣中心', '修正后货量']].copy()
adjusted_data.rename(columns={'修正后货量': '货量'}, inplace=True)
adjusted_data['货量类型'] = '修正后货量'
plot_data = pd.concat([comparison_data[['始发分拣中心', '到达分拣中心', '货量', '货量类型']], adjusted_data])
# 绘制柱状图
plt.figure(figsize=(20, 10))
sns.barplot(x='始发分拣中心', y='货量', hue='货量类型', data=plot_data, palette='viridis', ci=None) # ci=None 移除误差线
plt.title('Comparison of Cargo Volumes Before and After Adjustment Without Error Bars')
plt.xlabel('Start Sorting Center')
plt.ylabel('Cargo Volume')
plt.xticks(rotation=90) # 旋转x轴标签以便更好地阅读
plt.legend(title='货量类型')
plt.show()

```

附录 7

介绍：各分拣中心以及对应的货量变化率

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
# 读取Excel文件
file_path = r"C:\Users\20157\Desktop\货量数据.xlsx"
data = pd.read_excel(file_path)
# 计算变化率
data['变化率'] = data['新增货量'] / data['原始货量']
# 设置图形参数
plt.figure(figsize=(12, 8))
sns.barplot(x='到达分拣中心', y='变化率', data=data, palette='coolwarm')
plt.title('各到达分拣中心的货量变化率')
plt.xlabel('到达分拣中心')
plt.ylabel('变化率 (%)')
plt.xticks(rotation=45)
plt.tight_layout()
# 保存可视化结果到文件
visualization_path = r"C:\Users\20157\Desktop\新建文件夹(2)\Change_Rate_Visualization.png"
plt.savefig(visualization_path)
plt.show()

```

问题三：

附录 1

介绍：基于纯整数规划模型探究未来 30 天每个分拣中心每个班次出勤人数

```

import pandas as pd
from pulp import LpProblem, LpMinimize, LpVariable, lpSum, LpStatus
# 加载数据
data_path = r"C:\Users\20157\Desktop\货量数据.xlsx"
data = pd.read_excel(data_path)
shifts = ['00:00-08:00', '05:00-13:00', '08:00-16:00', '12:00-20:00',
'14:00-22:00', '16:00-24:00']
# 参数定义
efficiency_full = 25 # 正式工效率
efficiency_temp = 20 # 临时工效率
max_fulltime_workers = 60 # 每个分拣中心的最大正式工人数
# 初始化问题
problem = LpProblem("Minimize_Person_Days", LpMinimize)
# 决策变量
x_vars = LpVariable.dicts("X", [(center, date, shift) for center in
data['分拣中心'].unique() for date in data['日期'].unique() for shift in
shifts], 0, max_fulltime_workers, cat='Integer')
y_vars = LpVariable.dicts("Y", [(center, date, shift) for center in
data['分拣中心'].unique() for date in data['日期'].unique() for shift in
shifts], 0, cat='Integer')
# 目标函数
problem += lpSum([x_vars[c, d, s] + y_vars[c, d, s] for c in data['分拣中心
'].unique() for d in data['日期'].unique() for s in shifts])
# 约束条件
for center in data['分拣中心'].unique():
    for date in data['日期'].unique():
        problem += lpSum([x_vars[center, date, shift] for shift in
shifts]) <= max_fulltime_workers
        for shift in shifts:
            required_load = data[(data['分拣中心'] == center) & (data['日期
'] == date)][['货量']].iloc[0] / len(shifts)
            problem += x_vars[center, date, shift] * efficiency_full +
y_vars[center, date, shift] * efficiency_temp >= required_load
# 解决问题
problem.solve()
# 结果输出到CSV
results = []
for center in data['分拣中心'].unique():
    for date in data['日期'].unique():
        for shift in shifts:
            results.append({
                '分拣中心': center,
                '日期': date,
                '班次': shift,
                '正式工人数': x_vars[center, date, shift].varValue,
                '临时工人数': y_vars[center, date, shift].varValue
            })
results_df = pd.DataFrame(results)
results_df.to_csv(r"C:\Users\20157\Desktop\结果5.xlsx", index=False)
print("Optimization completed. Results saved to 'output_results.csv'.")

```

结果表

问题一：

表：SC12 未来 3 天货量预测结果

阶数（时间）	预测结果	阶数（时间）	预测结果
1	5325	16	5449
2	5359	17	5440
3	5387	18	5429
4	5411	19	5417
5	5430	20	5404
6	5446	21	5390
7	5457	22	5375
8	5466	23	5359
9	5472	24	5343
10	5475	25	5326
11	5475	26	5309
12	5474	27	5291
13	5470	28	5273
14	5465	29	5255
15	5458	30	5236